Agenda – Sessão 1 & 2

1) Recap
   - Sessão Anterior / Dúvidas
2) Today
   - Apresentação
     - Professor
     - Alunos
   - Calendário
   - Conteúdos
   - Avaliação
   - Linguagem
   - Classroom
     - Código
     - Todos os conteúdos publicados das sessões
   - Ambiente de Trabalho
   - Foundations
     - Sistema de Informação
     - Anatomia de um SI

2) Today (Cont)
   - IDE’s
   - VCS
   - Desafio da Sessão
     - Criar o seu próprio ambiente de desenvolvimento em Python

Forgot to Mention

- Awful Power Point Creator
- Feel free to contact:
  - mobile: +351 939 051 413
  - LinkedIn: https://www.linkedin.com/in/miguel-r-a17101149/
  - luismiguel.rodrigues@my.istec.pt

- What was the English mistake I did in this presentation ? Anyone noticed it ?

Calendário

1) Início das aulas do 2º Semestre 16/03/2026
2) Férias da Páscoa 30/03/2026 a 06/04/2026
3) Fim das aulas do 2º Semestre 12/06/2026
4) Outras atividades lectivas (Aulas de Compensação) 15/06/2026 a 20/06/2026
5) Exames do 2º Semestre 22/06/2026 a 31/07/2026 (Calendário específico a afixar oportunamente)

Avaliação - I

1) Todas as aulas são de natureza teórico-prática.
2) Estão previstas 60 horas de contato.
3) O tempo total de trabalho do aluno corresponde a 162 horas.
4) Numa segunda fase, de caraterísticas fundamentalmente aplicacionais, os alunos dverão adquirir familiaridade com um dispositivo RaspberryPi[2] e implementar aplicações usando a linguagem Python.
5) De acordo com o Regulamento de Funcionamento do ISTEC a avaliação é efetuada através de um exame escrito individual e obrigatório.
6) Na classificação final, vai ser considerado o projecto Final (em grupo, no máximo de 3; preferencialmente 2) .
7) CF = Valor do Exame *15/20 + Valor Trabalho*5/20[1]

1 – A confirmar pela Turma
2 – Confirmar Disponibilidade ou será proposto uma alternativa

CONTEÚDOS PROGRAMÁTICOS [1]

- 1. Introdução à linguagem Python
  - 1.1. Origens e evolução da linguagem
  - 1.2. Caraterísticas da linguagem
  - 1.3. Modos de execução de código
- 2. Plataforma de desenvolvimento
  - 2.1. IDEs para Python
  - 2.2. Instalação de plataformas de desenvolvimento
  - 2.3. Execução de código
- 3. Organização de execução de código Python
  - 3.1. Módulos e packages
  - 3.2. Names e namespaces
  - 3.3. Scopes
  - 3.4. Objetos e classes
- 4. Tipos de dados intrínsecos
  - 4.1. Tipos mutáveis e imutáveis
  - 4.2. Números
    - 4.2.1. Inteiros. Booleans, Floating-point, Complexos, Frações, strings e bytes
    - 4.2.1 Operações básicas
- 4.3. Tuples
- 4.4. Listas
- 4.5. Byte arrays
- 4.6. Set types
- 4.7. Dicionários
- 4.8. O módulo Collections.
- 5. Estruturas de decisão
  - 5.1. If/If… else/Ifl…elif
  - 5.2. Operador ternário
- 6. Estruturas de repetição
  - 6.1. Loop for
    - 6.1.1. Iterações em ranges
    - 6.1.2. Iterações em sequências
  - 6.2. Loop while
  - 6.3. Statements Break e Continue
- 8. Funções
  - 8.1. Conceito de função / 8.2. Utilidade das funções
  - 8.4. Scopes
  - 8.5. Passagem de argumentos
  - 8.6. Valores retornados
  - 8.7. Funções recursivas
  - 8.8. Funções anónimas
  - 8.9. Funções built-in
- 9. Programação Orientada a Objetos
  - 9.1. Conceitos introdutórios
  - 9.2. Classes em Python
  - 9.3. Inicializar instâncias
  - 9.4. Herança e composição
  - 9.5. Acesso a classe base
  - 9.6. Herança múltipla
  - 9.7. Classes e métodos static
  - 9.8. Private methods
  - 9.9. Operator overloading
  - 9.10. Polimorfismo

10. Projeto de programação em Python …..

1 - https://www.istec.pt/index.php/programacao-iv/

BIBLIOGRAFIA

1) COSTA, Ernesto. (2015). Programação em Python. Lisboa: FCA
2) BORGES, Luiz (2015); PYTHON para desenvolvedores, NOVATEC, S. Paulo.
3) BHASKAR, Chaudhary (2015); Tkinter GUI Application Development Blueprints, PACKT, Birmingham.
4) MARTINS, João. (2015). PYTHON – Introdução à programação utilizando múltiplos paradigmas. Lisboa: IST Press.
5) PERKINS, Jacob (2014); PYTHON 3 Text Processing with NLTK 3 Cookbook, PACKT, Birmingham.
6) VASCONCELOS, José (2015); PYTHON – Algoritmia e Programação Web, FCA, Lisboa.

Linguagem

1. Respeito pelos Outros
2. Always feel free to ask and add value

Ambiente

- Classrom code – **gqpivht5**

- Windows, Linux , Mac
- Python environments
  - https://www.python.org/downloads/
  - https://colab.research.google.com/notebooks/basic_features_overview.ipynb
    - Porque não funciona ;-)
  - Editores versus IDE
  - Editores: nano, notepad, vim ….
  - IDE: PyCharm, CoPilot , Pulsar ….

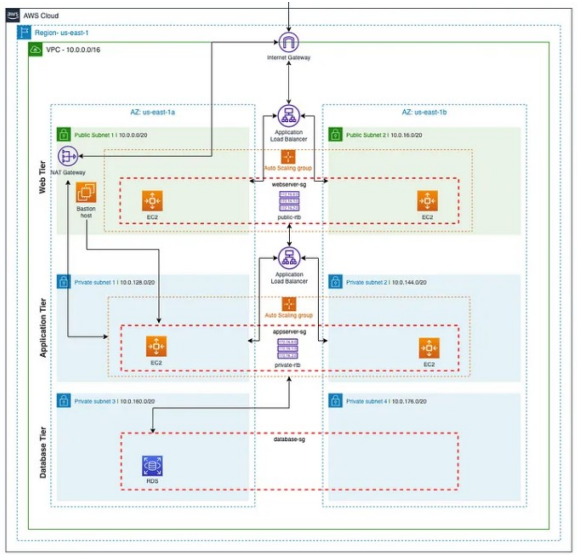
Sistema de Informação

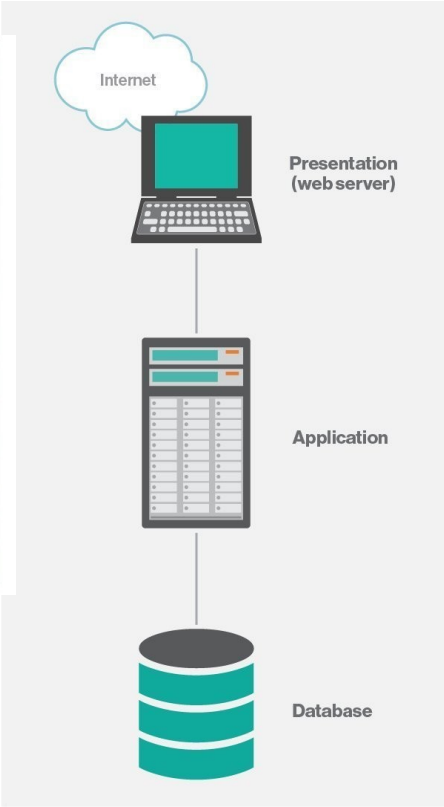
- Um sistema de informação (SI) é um conjunto de componentes e processos que trabalham juntos para recolher, armazenar, processar, transmitir e apresentar informações de forma eficiente e eficaz. O objetivo principal de um SI é fornecer informações precisas e atualizadas para apoiar as decisões e operações de uma organização.

- **Hardware**: Equipamentos físicos, como computadores/servidores, dispositivos de armazenamento e dispositivos de rede.
- **Software**: Programas e sistemas operativos que executam as funções do SI .
- **Bases de dados**: Um repositório de informações estruturadas ou não, organizadas que armazena e gere os **dados** do SI.
- **Processos**: Fluxos de trabalho e procedimentos que definem como os **dados** são recolhidos e processados.
- **Pessoas**: Os utilizadores do SI que utilizam as informações fornecidas pelo SI para tomar decisões e executar “”tarefas.

Anatomia de um SI

- 3 Níveis
- Conceitos
  - Host (Servidor)
  - Endpoint (Ponto de Rede)
  - Client/Server (Cliente / Servidor)
  - Linguagem programação
    - ex: Java, python,
  - Linguagem de marcação
    - Ex: Html, Markdown





Sistema de Informação

VCS's -1 [1]

1) Um sistema de controle de versões , na função prática da Engenharia de Software, é um software que tem a finalidade de gerir diferentes versões no desenvolvimento de um documento qualquer.

2) Esses sistemas são normalmente utilizados no desenvolvimento de software para controlar o histórico e desenvolvimento dos códigos-fontes e da documentação.

3) O desenvolvimento de software utiliza mais o Git[2] (com repositórios no GitHub[https://github.com/], Bitbucket [https://bitbucket.org/], GitLab, …)

1 - https://pt.wikipedia.org/wiki/Sistema_de_controle_de_vers%C3%B5es

2- https://pt.wikipedia.org/wiki/Git

“Versionamento”

1) É o processo de atribuir uma númeração única para indicar o estado de um programa de computador;
2) Esses números são geralmente atribuídos de forma crescente e indicam o desenvolvimento de actualizações.

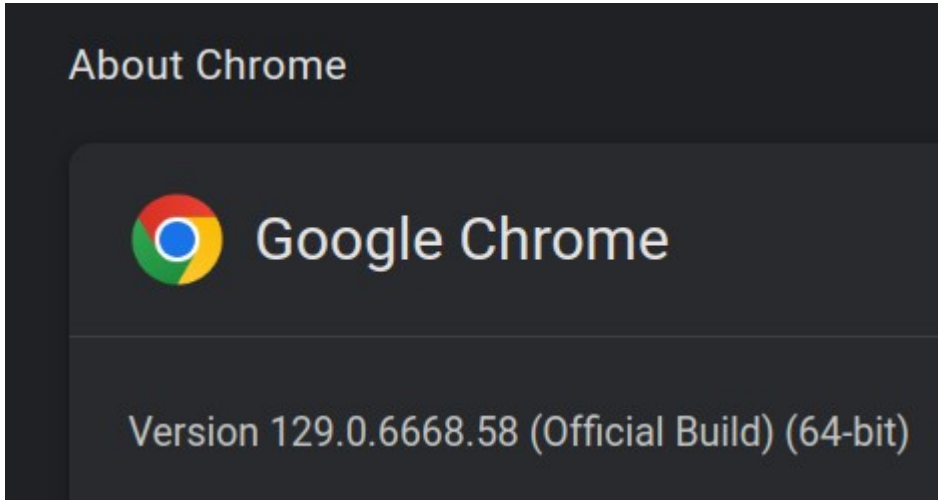
VCS’s – “Versionamento Semântico” (SemVer)

1) Usa uma sequência de três números (MAJOR.MINOR.PATCH);

2) São sempre levadas em conta se as mudanças são compatíveis ou não com versões anteriores. O número deve ser ser inclremental

3) MAJOR.MINOR.PATCH.”abcdef”

- MAJOR sempre que ocorrerem mudanças não compatíveis com versões anteriores;
- MINOR quando há adição de funcionalidade compatível com versões anteriores;
- PATCH para todos os outros tipos de mudanças, incluindo ajustes de “bugs”, também compatíveis com versões anteriores.
- Extensível….
  - A presença de uma “etiqueta” (-alpha, -beta)
  - “subpatch”



Um ser Humano

1) Linus “Benedict” Torvalds
   - Git foi inicialmente projetado para a kernel Linux[1][2];

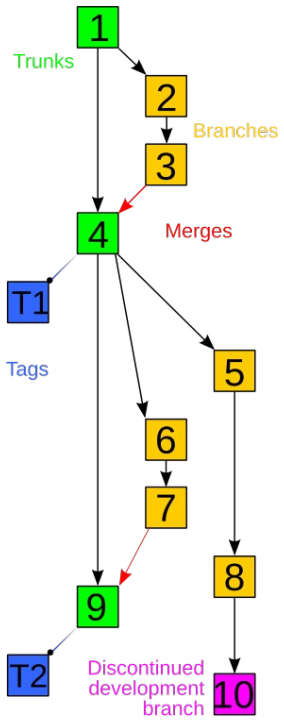
1 - https://en.wikipedia.org/wiki/Linux_kernel
2 - https://en.wikipedia.org/wiki/Android_(operating_system)

Funções VCS

- “Time Machine”: Se “ der asneira” vamos voltar a uma versão anterior;
- Compare & Check : O que mudou entre versão A e B ;
- Why ?
  - Version History: Mantenha um registo completo de todas as mudanças;
    - Quem/Quando/O Quê / Porquê /Em que versão

Git Conceitos

- Commit
- Remotes
- Push/Pull
- Branches
- Merge
- Tags



Git Conceitos – Remotes

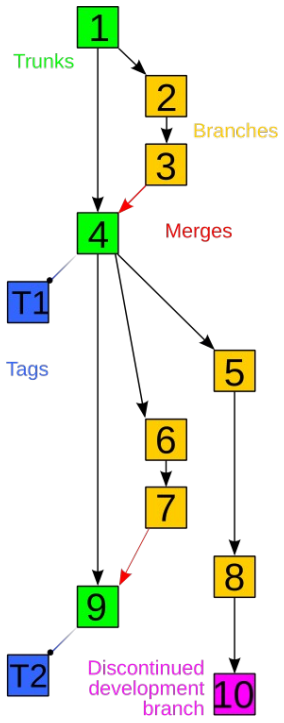
- **Remote** é uma referência a um repositório Git externo;
- É como um link para outro repositório (gitserver).

Git Conceitos – Push/Pull

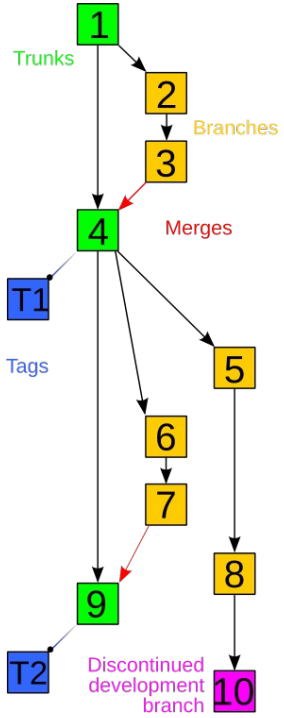
- Sincronizar mudanças entre o repositório local e um repositório remoto
  - Push - Envia as alterações locais (commits) para um repositório remoto.
  - Pull – Recebe as alterações mais recentes de um repositório remoto.

Git Conceitos – Merge

- “Juntar” o histórico de dois branches em um único histórico;
- Incorporar duas “timelines ”.



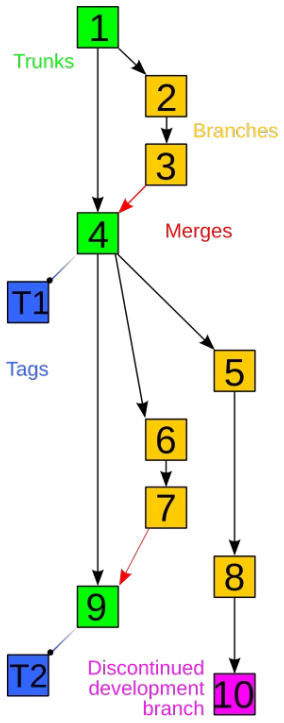
Git Conceitos – Branches



- Branch’s são como linhas paralelas de desenvolvimento
- Trabalhar em diferentes funcionalidades
  - De forma isolada
    - Sem afetar o código principal.
- Experimentar novas ideias ou trabalhar em várias coisas ao mesmo tempo.

Git Conceitos – Tags

- “Bandeiras” que se colocam em alguns commits
- São usados para identificar pontos importantes no histórico das versões



Ambientes integrados de desenvolvimento (IDE's)

- IDE é uma sigla para **Integrated Development Environment**
- É uma aplicação que reúne diversas ferramentas e funcionalidades para facilitar o processo de desenvolvimento de software.

- Main:
  - **Editor de código**: Um editor de texto avançado com destaque de **sintaxe**, **autocomplete** , **indentação automática** e outras funcionalidades.
  - **Compilador e interpretador**
  - **Debug**: Permite executar o código step by step , inspecionar variáveis.
  - **Integração com outras ferramentas**:
    - SVC (Git), ferramentas de build;
    - Outras ferramentas de desenvolvimento.
- Why
  - **Aumento da produtividade**: Automatizar tarefas repetitivas.
  - **Melhora da qualidade do código**:
  - **Facilidade de uso**
  - Exemplos :
    - Visual Studio Code(VSC)
    - IntelliJ (Java).
    - Eclipse (Java, Others)
    - PyCharm(Python)
    - Atom/Pulsar

Criar o seu próprio ambiente de desenvolvimento local em Python

- Instalar no seu ambiente local
  - VCS (Git)
  - Python
    - (3.X please)
  - Create personal git remote account (eg: GitHub, Bitbucket or any other)
  - IDE